

Reimbursements Platform Blueprint

Campo	Valor
Documento	RPC-002
Nombre	AI Collaboration Charter
Versión	1.0
Estado	Draft
Clasificación	Blueprint
Autor	Architecture Office
Responsable	Chief Software Architect
Audiencia	Arquitectos, Líder Técnico, Líder Funcional, Desarrolladores, Revisores Técnicos, IA involucradas
Última actualización	2026-07-24

1. Propósito

Este documento define el **modelo operativo oficial de colaboración entre personas e inteligencias artificiales** dentro del Reimbursements Platform Blueprint.

Su propósito es transformar los principios constitucionales establecidos por:

RPC-001 – AI Project Constitution

en reglas concretas de trabajo cotidiano.

RPC-002 establece:

- Roles.
- Responsabilidades.
- Handoffs.
- Flujos de colaboración.
- Revisión.
- Aprobación.

- Escalamiento.
- Uso de contexto.
- Preparación de tareas.
- Manejo de resultados.
- Separación de responsabilidades.
- Interacción entre diferentes herramientas de IA.

Este documento responde principalmente:

¿Cómo deberán colaborar humanos e IA durante el proyecto respetando la autoridad, límites y principios establecidos por la Constitución?

2. Alcance

RPC-002 aplica a cualquier actividad del proyecto en la que participe una herramienta de inteligencia artificial.

Incluye:

- Análisis.
- Discovery.
- Arquitectura.
- Diseño.
- Documentación.
- Generación de propuestas.
- Investigación.
- Implementación asistida futura.
- Testing.
- Revisión.
- Auditoría.
- Diagramación.
- Prompt Engineering.
- Mantenimiento documental.

Aplica tanto a interacciones individuales como a flujos en los que participen múltiples personas o herramientas de IA.

3. Fuera de Alcance

RPC-002 no define:

- Autoridad constitucional de IA.
- Gobierno arquitectónico completo.

- Proceso formal de excepciones.
- Bounded Contexts.
- Microservicios.
- Arquitectura concreta.
- Estándares técnicos de implementación.
- Herramientas obligatorias de IA.
- Prompts específicos.

La autoridad constitucional reside en:

RPC-001 – AI Project Constitution

El gobierno arquitectónico formal será responsabilidad de:

RPC-003 – Architecture Governance

4. Relación con RPC-001

La relación entre ambos documentos es:

RPC-001
AI Project Constitution
Define principios, límites y autoridad

↓

RPC-002
AI Collaboration Charter
Define la operación cotidiana

RPC-002 no podrá relajar ni contradecir una regla constitucional establecida en RPC-001.

5. Principio Operativo Fundamental

La colaboración seguirá:

Humanos deciden y responden; la IA acelera, analiza, propone y verifica.

La participación de IA deberá aumentar la capacidad del equipo sin diluir la responsabilidad.

6. Modelo General de Colaboración

El flujo general será:

```
Necesidad
↓
Context Preparation
↓
AI-Assisted Analysis
↓
Proposal / Artifact Draft
↓
Human Review
↓
Decision / Approval
↓
Execution
↓
Validation
↓
Documentation
↓
Audit / Feedback
```

No todas las actividades utilizarán cada etapa.

El flujo deberá adaptarse proporcionalmente a:

- Riesgo.
- Impacto.
- Reversibilidad.
- Complejidad.

7. Roles Humanos

El modelo reconoce inicialmente los siguientes roles humanos:

```
Project / Functional Leadership
Chief Software Architect responsibility
Technical Leadership
```

Developers
Reviewers / Specialists

Las personas específicas podrán cambiar.

Las responsabilidades deberán permanecer explícitas.

8. Líder Funcional

Responsabilidades

El Líder Funcional será responsable de:

- Proporcionar conocimiento del negocio.
- Validar interpretación funcional.
- Resolver ambigüedades.
- Identificar excepciones.
- Aprobar conclusiones funcionales cuando corresponda.
- Mantener alineación con stakeholders.

La IA no deberá sustituir la validación funcional de este rol.

9. Chief Software Architect

Responsabilidad

El rol de Chief Software Architect será responsable de:

- Mantener coherencia arquitectónica.
- Guiar análisis.
- Evaluar trade-offs.
- Preparar decisiones.
- Mantener la disciplina de fases.
- Identificar necesidad de ADR.
- Revisar propuestas técnicas.
- Coordinar gobierno del Blueprint.
- Diseñar especificaciones para implementación asistida.

Una IA podrá operar como asistente bajo este rol conceptual.

La autoridad formal continúa siendo humana.

10. Líder Técnico

Responsabilidades

El Líder Técnico deberá:

- Traducir arquitectura aprobada a coordinación de implementación.
- Detectar problemas prácticos.
- Revisar factibilidad.
- Mantener consistencia entre desarrolladores.
- Escalar contradicciones arquitectónicas.
- Participar en revisiones técnicas.

No deberá introducir cambios arquitectónicos significativos de manera implícita durante implementación.

11. Desarrolladores

Responsabilidades

Los desarrolladores deberán:

- Comprender la solución implementada.
 - Utilizar IA como asistencia, no como sustituto de comprensión.
 - Revisar código generado.
 - Ejecutar pruebas.
 - Cumplir Standards.
 - Detectar inconsistencias.
 - Reportar decisiones implícitas.
 - Mantener trazabilidad cuando corresponda.
-

12. Revisores y Especialistas

Dependiendo de la necesidad podrán participar:

- Security specialists.
- Database specialists.
- Infrastructure specialists.
- QA specialists.
- Business specialists.

La IA podrá complementar sus análisis.

No deberá presentarse como sustituto de expertise requerido cuando el riesgo exija revisión especializada.

13. Roles de Inteligencia Artificial

El modelo permitirá diferentes roles conceptuales de IA.

Los principales serán:

```
Architecture Assistant
Implementation Assistant
Reviewer
Auditor
Research Assistant
Documentation Assistant
Prompt Engineering Assistant
```

Un mismo modelo podrá desempeñar distintos roles en momentos diferentes.

El rol activo deberá quedar claro.

14. Architecture Assistant

Responsabilidades

Podrá:

- Analizar problemas.
- Formular preguntas.
- Evaluar alternativas.
- Identificar trade-offs.
- Detectar riesgos.
- Proponer estructuras.
- Elaborar drafts arquitectónicos.
- Revisar coherencia.

No podrá:

- Aprobar decisiones.
- Saltar Discovery.
- Inventar requisitos.
- Declarar una propuesta como arquitectura definitiva.

15. Implementation Assistant

Responsabilidades futuras

Una vez habilitada la fase correspondiente podrá:

- Generar código.
- Proponer refactorizaciones.
- Crear tests.
- Preparar configuraciones.
- Crear scaffolding.
- Aplicar especificaciones aprobadas.

Deberá consumir arquitectura y Standards.

No deberá redefinirlos durante la implementación.

16. Reviewer

Una IA en rol Reviewer podrá:

- Comparar un artefacto con reglas existentes.
- Identificar defectos.
- Detectar inconsistencias.
- Clasificar hallazgos.
- Proponer correcciones.

La revisión asistida no constituye aprobación.

17. Auditor

Una IA en rol Auditor deberá buscar evidencia de:

- Cumplimiento.
- Incumplimiento.
- Drift.
- Riesgos.
- Contradicciones.

Deberá mantener independencia conceptual respecto de la tarea de creación cuando sea posible.

18. Research Assistant

Podrá:

- Investigar documentación.
- Comparar tecnologías.
- Resumir fuentes.
- Identificar alternativas.

Deberá distinguir conocimiento externo de decisiones internas.

19. Documentation Assistant

Podrá:

- Generar drafts.
- Aplicar Templates.
- Mejorar consistencia.
- Actualizar referencias.
- Detectar metadata faltante.

No deberá modificar decisiones durante una tarea puramente editorial.

20. Prompt Engineering Assistant

Podrá:

- Diseñar prompts.
- Revisar prompts.
- Crear criterios de validación.
- Detectar ambigüedad.
- Reducir Prompt Drift.

Los PRM oficiales deberán respetar TPL-003.

21. Modelo de Herramientas Inicial

El contexto fundacional establece inicialmente:

ChatGPT
Chief Software Architect / Architecture Assistant

Claude
Implementation Assistant

Esta asignación representa una estrategia inicial.

No crea dependencia permanente con proveedores específicos.

Podrá evolucionar de acuerdo con:

- Calidad.
- Capacidades.
- Seguridad.
- Costos.
- Madurez del equipo.

22. ChatGPT — Responsabilidad Inicial

ChatGPT será utilizado principalmente para:

- Arquitectura.
- DDD guidance.
- Gobierno.
- Documentación.
- Auditoría.
- Diseño de prompts.
- Análisis de trade-offs.
- Revisión de propuestas.

Toda salida seguirá requiriendo validación humana conforme a RPC-001.

23. Claude — Responsabilidad Inicial

Claude podrá utilizarse principalmente para:

- Implementación asistida.
- Análisis de código.
- Generación de propuestas detalladas.
- Refactorización.
- Creación de artefactos técnicos.

- Documentación secundaria.

Cuando se utilice para tareas arquitectónicas, sus resultados deberán tratarse como propuestas sujetas al mismo gobierno.

24. Tool Neutrality

Aunque existan asignaciones iniciales, los documentos deberán expresar responsabilidades principalmente mediante roles.

Preferencia:

Implementation Assistant

sobre:

Tool X

cuando el proveedor concreto no sea relevante.

25. Modelo de Handoff

Un handoff ocurre cuando la responsabilidad de una actividad pasa de una persona o herramienta a otra.

Todo handoff importante deberá entregar:

1. Objetivo.
 2. Contexto.
 3. Inputs.
 4. Fuentes autoritativas.
 5. Restricciones.
 6. Estado del trabajo.
 7. Resultado esperado.
 8. Criterios de aceptación.
 9. Decisiones ya tomadas.
 10. Incertidumbres abiertas.
-

26. Handoff Incompleto

No deberá realizarse un handoff importante basado únicamente en:

“Continúa con esto”

cuando el receptor no disponga del contexto necesario.

La continuidad deberá depender de conocimiento explícito, no de memoria accidental.

27. Context Package

Para cada tarea importante deberá prepararse conceptualmente un:

Context Package

Un Context Package podrá contener:

- Project identity.
 - Phase.
 - Task.
 - Relevant Blueprint documents.
 - ADR.
 - Standards.
 - Inputs.
 - Constraints.
 - Acceptance Criteria.
-

28. Context Package Mínimo

Para una tarea de bajo impacto podrá bastar:

Objective
Relevant input
Applicable rule
Expected output

Para tareas críticas se requerirá mayor profundidad.

29. Context Package como Contrato

El Context Package funciona como un contrato operativo entre:

Requester
↓
AI / Human Executor

Permite evitar:

- Supuestos ocultos.
- Diferencias de contexto.
- Reinterpretación.
- Tareas fuera de alcance.

30. Preparación de Tareas para IA

Antes de asignar una tarea relevante deberá verificarse:

- ¿El objetivo es claro?
- ¿La fase lo permite?
- ¿Existe suficiente contexto?
- ¿Las reglas relevantes están disponibles?
- ¿La salida esperada está definida?
- ¿La IA tiene autoridad adecuada?
- ¿La tarea requiere revisión especializada?

31. Clasificación por Riesgo

Las tareas asistidas podrán clasificarse conceptualmente como:

Low Risk
Medium Risk
High Risk
Critical

La clasificación no necesita formalizarse para cada interacción cotidiana.

Deberá utilizarse cuando el impacto justifique controles adicionales.

32. Low Risk

Ejemplos:

- Corrección editorial.
- Formato.
- Resumen interno.
- Generación de ideas no decisorias.

Podrán requerir revisión ligera.

33. Medium Risk

Ejemplos:

- Documentación técnica.
- Propuestas locales.
- Tests.
- Refactorización limitada.

Requieren revisión humana antes de incorporación.

34. High Risk

Ejemplos futuros:

- Arquitectura.
- Seguridad.
- Persistencia.
- Integración.
- Cambios transversales.
- Decisiones difíciles de revertir.

Deberán tener:

- Contexto amplio.
- Alternativas.
- Review.
- Approval.

35. Critical

Ejemplos futuros:

- Producción.
- Datos sensibles.
- Acciones destructivas.
- Migraciones críticas.
- Seguridad de alto impacto.

Requerirán controles reforzados definidos por documentos posteriores.

36. Flujo de Análisis

Para una tarea de análisis:

Human
Provides problem and context

↓

AI
Analyzes

↓

AI
Separates:
Facts
Assumptions
Questions
Risks
Options

↓

Human
Validates

↓

Knowledge captured if relevant

37. Flujo de Propuesta Arquitectónica

Cuando la fase lo permita:

```
graph TD
    Problem --> Context
    Context --> AI[AI proposes alternatives]
    AI --> Trade[Trade-off analysis]
    Trade --> Review[Human / Architecture Review]
    Review --> Decision
    Decision --> ADR[ADR when required]
```

La IA no deberá saltar desde:

Problem

directamente hacia:

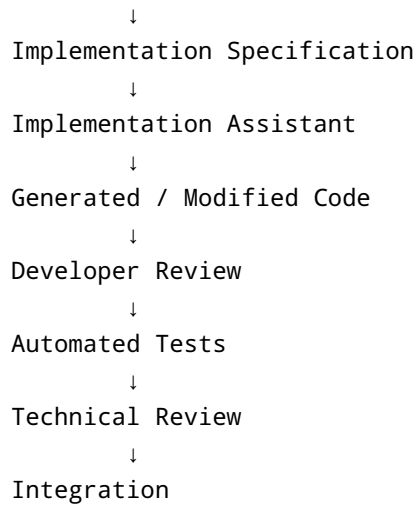
Implementation

en decisiones relevantes.

38. Flujo de Implementación Asistida

Cuando esté habilitado:

```
graph TD
    ApprovedArch[Approved Architecture] --> ApprovedStand[Approved Standards]
```

39. Specification Before Generation

Para implementación no trivial deberá existir una especificación suficiente antes de solicitar código.

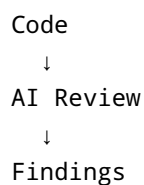
Podrá incluir:

- Responsibility.
- Inputs.
- Behavior.
- Constraints.
- Interfaces.
- Error handling.
- Acceptance Criteria.
- Tests expected.

Esto reduce generación basada en inferencias.

40. Flujo de Code Review Asistido

Futuro modelo:



↓
Developer assessment
↓
Correction
↓
Human / automated validation

La IA podrá complementar revisiones, no sustituirlas.

41. Flujo de Documentación

Need
↓
Relevant sources
↓
AI Draft
↓
Human Review
↓
Revision
↓
Approval
↓
Blueprint update

Este es el flujo utilizado actualmente para la construcción del Blueprint.

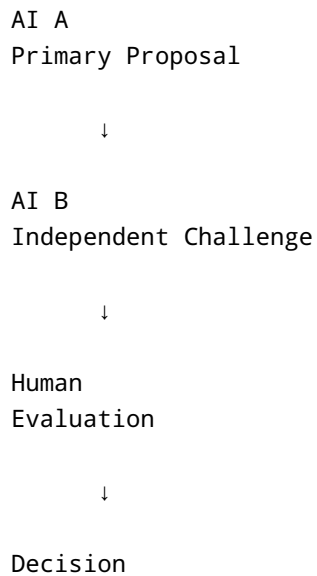
42. Flujo de Auditoría

Approved Rules
↓
Artifact
↓
AI Audit
↓
Evidence-based findings
↓
Human Review
↓
Action

Una auditoría deberá basarse en reglas existentes.

43. Flujo Multi-IA

Para tareas importantes podrá utilizarse:



Este modelo es opcional.

No deberá introducirse automáticamente para todas las tareas.

44. Separation of Creation and Review

Cuando el riesgo lo justifique, será preferible separar:

Creator

de:

Reviewer

Esto puede realizarse mediante:

- Diferentes personas.
- Diferentes IA.
- Diferentes sesiones.
- Combinación de los anteriores.

45. Same-Model Review

Un mismo modelo podrá revisar su trabajo cuando no exista otra alternativa.

La revisión deberá instruirse explícitamente para adoptar una perspectiva crítica.

No se considerará equivalente a revisión independiente.

46. Human Review Depth

La profundidad de revisión dependerá de:

- Riesgo.
- Complejidad.
- Reversibilidad.
- Familiaridad del equipo.
- Grado de generación automática.

Más automatización no deberá significar menos responsabilidad.

47. Review Before Approval

No deberá utilizarse:

Generated
↓
Approved

El flujo correcto será:

Generated
↓

Reviewed

↓

Corrected if necessary

↓

Approved

48. Acceptance Criteria

Toda tarea no trivial debería tener criterios claros de finalización.

Ejemplo conceptual:

The task is complete when:

- ...
- ...

Esto permite evaluar resultados sin depender únicamente de impresión subjetiva.

49. Done vs Approved

Deberá distinguirse:

Done

de:

Approved

Una IA puede completar una tarea.

Sólo la autoridad correspondiente puede aprobar un artefacto cuando el gobierno lo requiera.

50. Feedback Loop

Las correcciones realizadas después de trabajo asistido deberán utilizarse para mejorar:

- Context Packages.
- Prompts.
- Standards.
- Checklists.
- Guides.
- Model selection.

El objetivo será evitar errores repetitivos.

51. Error Taxonomy

Los errores de colaboración podrán clasificarse conceptualmente como:

```
Context Error
Requirement Error
Reasoning Error
Architecture Error
Implementation Error
Documentation Error
Tool Error
Review Error
```

Esta clasificación ayudará a identificar controles adecuados.

52. Context Error

Ocurre cuando el resultado es incorrecto porque la IA recibió:

- Información insuficiente.
- Información obsoleta.
- Contexto equivocado.
- Fuentes contradictorias no identificadas.

La corrección deberá mejorar el Context Package, no únicamente el resultado.

53. Requirement Error

Ocurre cuando:

- Se inventó un requisito.
- Se interpretó mal una necesidad.
- El requisito era ambiguo.

La resolución deberá involucrar a la fuente funcional correspondiente.

54. Reasoning Error

Ocurre cuando la información era suficiente, pero la conclusión fue incorrecta.

Podrá requerir:

- Revisión adicional.
 - Nueva estrategia de prompt.
 - Otro modelo.
 - Validación especializada.
-

55. Architecture Error

Ocurre cuando una propuesta viola:

- Principios.
- ADR.
- Standards.
- Fases.
- Responsabilidades.

Deberá ser bloqueada antes de implementación.

56. Implementation Error

Ocurre cuando el código o configuración:

- No cumple especificación.
- Tiene defectos.
- Introduce vulnerabilidad.

- Viola Standards.

Deberá corregirse mediante los mecanismos técnicos correspondientes.

57. Documentation Error

Incluye:

- Estado incorrecto.
 - Metadata incorrecta.
 - Contradicciones.
 - Referencias obsoletas.
 - Knowledge Drift.
-

58. Tool Error

Ocurre cuando una herramienta:

- Devuelve información incorrecta.
- Falla.
- Utiliza datos obsoletos.
- Produce resultados parciales.

La IA deberá reconocer la limitación.

59. Review Error

Ocurre cuando una revisión no detecta un problema que debería haber identificado.

Podrá justificar mejora de:

- Checklists.
 - Prompts.
 - Independencia de revisión.
 - Expertise requerido.
-

60. Escalamiento

La IA deberá escalar conceptualmente una situación cuando:

- Falte una decisión humana.
- Exista contradicción normativa.
- Exista incertidumbre funcional material.
- El riesgo exceda su autoridad.
- Se requiera una excepción.
- Falte expertise especializado.

Escalar significa:

Identificar claramente qué debe resolver un humano o proceso superior.

61. No Escalation by Convenience

La IA no deberá escalar todo problema pequeño para evitar razonar.

Deberá resolver autónomamente dentro del contexto cuando:

- La información sea suficiente.
 - Exista una regla clara.
 - La tarea esté dentro de alcance.
-

62. Clarification Strategy

Cuando sea necesario aclarar una tarea, las preguntas deberán buscar información de alto impacto.

Prioridad:

1. Requisito desconocido.
2. Restricción crítica.
3. Outcome esperado.
4. Decisión humana faltante.

No deberán solicitarse detalles irrelevantes.

63. Assumption Strategy

Cuando una tarea pueda continuar razonablemente con un supuesto reversible:

1. Declarar el supuesto.
2. Continuar.
3. Evitar convertirlo en decisión permanente.

Cuando el supuesto pueda afectar arquitectura o negocio significativamente deberá solicitarse validación.

64. Research Workflow

Para investigación:

```
Question
↓
Authoritative sources first
↓
Multiple sources when needed
↓
Evidence extraction
↓
Comparison
↓
Project-context interpretation
↓
Recommendation if requested
```

Una recomendación externa deberá adaptarse al contexto.

65. Technical Comparison

Una IA no deberá comparar tecnologías únicamente por features.

Deberá considerar:

- Project constraints.
- Team maturity.
- Operability.
- Complexity.
- Costs.

- Ecosystem.
 - Reversibility.
 - Strategic fit.
-

66. Decision Brief

Antes de una decisión relevante podrá prepararse un:

Decision Brief

que contenga:

- Problem.
- Context.
- Drivers.
- Alternatives.
- Trade-offs.
- Recommendation.
- Open questions.

Cuando la decisión sea aceptada y significativa podrá evolucionar hacia ADR.

67. Architecture Review Package

Para revisiones arquitectónicas podrá prepararse:

Architecture Review Package

con:

- Problem statement.
- Proposed solution.
- Diagrams.
- Relevant ADR.
- Risks.
- Alternatives.
- Open issues.

RPC-003 definirá posteriormente el proceso formal.

68. Implementation Handoff Package

Cuando se llegue a implementación, un handoff desde arquitectura deberá incluir suficiente información para evitar reinterpretación.

Podrá contener:

- Capability / scope.
- Approved architecture.
- Relevant ADR.
- Applicable Standards.
- Interfaces.
- Constraints.
- Acceptance Criteria.
- Known risks.

69. Review Handoff Package

Al solicitar una revisión deberá incluirse:

- Artifact.
- Purpose.
- Relevant rules.
- Review scope.
- Expected depth.
- Known exceptions.

Esto evita revisiones genéricas sin contexto.

70. AI Output Classification

Una salida podrá clasificarse como:

```
Exploration
Draft
Proposal
Recommendation
Review Finding
Audit Finding
Implementation Artifact
```

Esta clasificación deberá usarse cuando ayude a evitar ambigüedad.

71. Exploration

Contenido destinado a ampliar opciones.

No representa intención de adopción.

72. Draft

Primer artefacto estructurado sujeto a cambios.

73. Proposal

Solución concreta sugerida para evaluación.

74. Recommendation

Alternativa preferida después de análisis.

No implica aprobación.

75. Review Finding

Observación derivada de revisar un artefacto contra criterios.

76. Audit Finding

Hallazgo respaldado por una regla o evidencia verificable.

77. Implementation Artifact

Código, configuración, script u otro resultado ejecutable generado durante una fase autorizada.

78. Decision Language

Las IA deberán evitar frases como:

"We will use..."

antes de la aprobación.

Preferir:

"I recommend..."

"The proposal is..."

"Alternative A is preferred because..."

Después de aprobación podrá utilizarse lenguaje decisorio citando la fuente correspondiente.

79. Documentation Language

La documentación oficial deberá distinguir claramente:

Proposed

de:

Approved

La redacción deberá reflejar el estado real.

80. Prompt Lifecycle

Un prompt podrá seguir:

Ad-hoc prompt
↓
Repeated useful pattern
↓
Candidate PRM
↓
Draft
↓
Review
↓
Approved PRM

No deberá formalizarse prematuramente cada interacción.

81. PRM Usage

Un PRM Approved deberá utilizarse cuando:

- La tarea corresponda a su alcance.
- La repetibilidad sea importante.
- El proceso requiera control.

Podrá adaptarse únicamente dentro de los límites permitidos por el propio PRM.

82. Prompt Customization

La personalización de un prompt deberá separar:

Stable instructions

de:

Task-specific context

Esto mejora reutilización.

83. Model Selection

La selección de IA podrá considerar:

- Type of task.
- Context capacity.
- Reasoning quality.
- Code capabilities.
- Tool support.
- Cost.
- Security.
- Latency.

No deberá utilizarse una herramienta para toda tarea únicamente por familiaridad.

84. Model Benchmarking

Para tareas críticas recurrentes podrán compararse modelos mediante casos representativos.

La evaluación deberá utilizar criterios reales del proyecto.

85. Cost Awareness

El uso de IA deberá considerar costos cuando sea relevante.

Sin embargo:

Menor costo no justifica menor calidad en tareas críticas.

El nivel de modelo y contexto deberá ser proporcional al valor y riesgo de la tarea.

86. Efficiency

Deberá evitarse:

- Repetir análisis ya documentados.
- Cargar contexto irrelevante.
- Rehacer artefactos aprobados.
- Utilizar múltiples IA sin beneficio.
- Generar documentación redundante.

87. Knowledge Reuse

Antes de generar nueva información deberá consultarse el conocimiento oficial disponible.

Flujo:

Need
↓
Search Blueprint
↓
Reuse existing knowledge
↓
Create new knowledge only if needed

88. Single Source of Truth

Si una respuesta de IA contradice un documento Approved:

Approved document wins

hasta que exista proceso formal para revisar la regla.

89. Conversation Memory

La memoria de una conversación podrá utilizarse como comodidad operativa.

No deberá sustituir el Blueprint.

Cuando la continuidad sea importante a largo plazo, el conocimiento deberá documentarse.

90. Session Handoff

Cuando una sesión de IA finalice y otra deba continuar, la continuidad deberá lograrse mediante:

- Blueprint.
- Context Package.

- State summary.
- Relevant artifacts.

No mediante expectativa de memoria invisible.

91. State Summary

Para trabajos prolongados podrá utilizarse un resumen operativo con:

Current phase
Current artifact
Approved decisions
Open questions
Next authorized action

Este resumen no reemplaza documentos oficiales.

92. Human Override

La autoridad humana podrá rechazar una recomendación de IA.

La IA deberá adaptarse a la decisión.

Cuando el override cambie una regla o arquitectura previamente aprobada deberá seguirse el gobierno documental correspondiente.

93. Human Instruction vs Blueprint

Si una instrucción humana contradice una regla vigente:

1. La contradicción deberá señalarse.
2. Deberá determinarse si se trata de una excepción o cambio.
3. La modificación deberá gobernarse cuando corresponda.

Esto protege al proyecto contra cambios accidentales.

94. Collaboration Anti-Patterns

El proyecto evitará los siguientes antipatrones.

AI Autopilot

Permitir que una IA tome una tarea compleja desde necesidad hasta producción sin controles.

Blind Delegation

Asignar una tarea que el solicitante no puede evaluar adecuadamente.

Context Starvation

Solicitar resultados complejos con información insuficiente.

Context Flooding

Entregar indiscriminadamente todo el proyecto sin relevancia.

Copy-Paste Engineering

Incorporar resultados sin comprenderlos.

Architecture by Prompt

Permitir que una instrucción aislada defina decisiones que deberían gobernarse.

Review Theater

Solicitar una revisión que no cambia realmente la posibilidad de aprobación.

AI Consensus Fallacy

Asumir que varias IA coincidentes prueban que una solución es correcta.

Human Rubber Stamp

Reducir la revisión humana a aprobar automáticamente resultados generados.

Prompt Cargo Cult

Aplicar técnicas complejas de prompting sin necesidad demostrada.

Tool Loyalty

Mantener una herramienta aunque otra resulte más apropiada para una tarea.

95. Collaboration Quality Attributes

El modelo deberá optimizar:

- Correctitud.
- Trazabilidad.
- Velocidad.
- Comprensión.
- Calidad arquitectónica.
- Seguridad.
- Repetibilidad.
- Mantenibilidad.
- Transferencia de conocimiento.

La velocidad deberá optimizarse sin sacrificar los demás atributos críticos.

96. Success Metrics Conceptuales

La colaboración será saludable cuando:

- Los errores se detecten antes.
- Las decisiones tengan mejor evidencia.
- El equipo comprenda lo generado.
- La documentación permanezca coherente.
- Exista menos trabajo repetitivo.
- Las revisiones sean más efectivas.
- El Blueprint se mantenga actualizado.

- La IA no se convierta en cuello de botella ni autoridad informal.

Las métricas concretas podrán definirse posteriormente si aportan valor.

97. Knowledge Transfer

La IA deberá utilizarse para aumentar la comprensión del equipo.

Cuando genere una solución compleja debería poder explicar:

- Concepto.
- Razón.
- Trade-off.
- Riesgo.

El objetivo no será únicamente producir artefactos.

Será mejorar la capacidad colectiva del equipo.

98. Learning Loop

La colaboración podrá seguir:

```
Task
↓
AI Assistance
↓
Human Review
↓
Outcome
↓
Lessons
↓
Improved Blueprint / Prompt / Practice
```

La evolución deberá convertir experiencia en conocimiento reutilizable.

99. Governance Escalation

Los asuntos que involucren:

- Cambios arquitectónicos.
- Excepciones.
- Conflictos de autoridad.
- Incumplimiento.
- Gates.

deberán escalarse al modelo definido posteriormente por:

RPC-003 – Architecture Governance

RPC-002 no resuelve por sí mismo dichos procesos.

100. Colaboración durante Business Discovery

Cuando se habilite Business Discovery, la IA podrá:

- Preparar preguntas.
- Organizar sesiones.
- Extraer reglas.
- Detectar contradicciones.
- Construir glosarios provisionales.
- Identificar escenarios faltantes.

No deberá:

- Inventar reglas.
 - Convertir de inmediato procesos en microservicios.
 - Sustituir validación funcional.
-

101. Colaboración durante Domain Discovery

Cuando se habilite Domain Discovery, la IA podrá:

- Analizar lenguaje.
- Proponer eventos.
- Identificar candidatos.
- Sugerir límites.

- Detectar conflictos de lenguaje.

Los resultados serán hipótesis hasta validación humana.

102. Colaboración durante Architecture Design

Cuando corresponda, la IA podrá:

- Generar alternativas.
- Analizar trade-offs.
- Preparar ADR.
- Revisar consistencia.
- Crear diagramas Draft.

No deberá saltar directamente a tecnologías sin drivers.

103. Colaboración durante Implementación

Cuando sea autorizada, la IA podrá:

- Generar código.
- Refactorizar.
- Generar tests.
- Crear documentación.
- Realizar análisis estático conceptual.

Siempre bajo arquitectura y Standards aprobados.

104. Colaboración durante Operación

En fases futuras podrá asistir en:

- Investigación de incidentes.
- Análisis de logs.
- Root Cause Analysis.
- Runbooks.
- Postmortems.

Los permisos y datos deberán cumplir reglas de seguridad.

105. Incident Assistance

Una IA podrá proponer acciones durante incidentes.

No deberá ejecutar acciones destructivas o de producción sin controles explícitos.

106. Postmortem Assistance

Podrá ayudar a:

- Construir timeline.
- Identificar contributing factors.
- Detectar gaps.
- Proponer mejoras.

No deberá buscar culpables individuales.

107. Document Ownership

La IA puede generar un documento.

No puede convertirse en su owner.

El Responsable deberá ser un rol humano.

108. Artifact Ownership

De forma similar:

Generated by AI

no equivale a:

Owned by AI

Todo artefacto incorporado deberá tener ownership humano.

109. Approval Matrix Conceptual

Tipo de salida	IA puede generar	Revisión humana	Aprobación formal
Exploración	Sí	Según necesidad	No
Draft documental	Sí	Sí	Según artefacto
Recomendación	Sí	Sí	Si implica decisión
ADR Draft	Sí	Sí	Sí
Arquitectura	Sí, como propuesta	Sí	Sí
Código futuro	Sí	Sí	Según proceso técnico
Standard Draft	Sí	Sí	Sí
Auditoría	Sí	Sí para acciones	No por IA
Business Rule	Sólo documentar evidencia	Sí	Autoridad funcional

110. Escalation Matrix Conceptual

Situación	Acción
Información faltante de bajo impacto	Declarar supuesto reversible
Requisito funcional desconocido	Escalar a liderazgo funcional
Conflicto arquitectónico	Escalar a Architecture Governance
Riesgo de seguridad	Solicitar revisión especializada
Contradicción documental	Detener dependencia y resolver
Tool failure	Declarar limitación y usar alternativa si existe
Cambio de decisión Approved	Iniciar proceso formal de cambio

111. Criterios de Calidad del Handoff

Un buen handoff deberá permitir que el receptor responda:

1. ¿Qué debo hacer?
2. ¿Por qué?
3. ¿Qué contexto necesito?

4. ¿Qué reglas no debo romper?
 5. ¿Qué ya está decidido?
 6. ¿Qué sigue abierto?
 7. ¿Qué salida debo producir?
 8. ¿Cómo sabremos que está correcta?
-

112. Criterios de Calidad de Colaboración

El modelo será adecuado si:

- Las responsabilidades son claras.
 - La autoridad humana permanece explícita.
 - Los handoffs no dependen de memoria informal.
 - Las IA trabajan con contexto gobernado.
 - Las revisiones son proporcionales al riesgo.
 - Los resultados pueden rastrearse.
 - Los errores producen aprendizaje.
 - Las herramientas pueden cambiar sin romper el modelo.
-

113. Criterios de Aceptación de RPC-002

RPC-002 podrá pasar a Approved cuando:

- Los roles humanos sean aceptados.
 - Los roles de IA sean aceptados.
 - El modelo de handoff sea validado.
 - El uso de Context Packages sea aprobado.
 - Los flujos de análisis, arquitectura, documentación e implementación futura sean aceptados.
 - La separación entre generación, revisión y aprobación sea clara.
 - El modelo Multi-IA sea considerado apropiado.
 - El modelo de escalamiento sea aceptado.
 - Los antipatrones sean validados.
 - La relación con RPC-001 sea correcta.
 - No existan contradicciones conocidas con RPC-001, RPC-000 o BOOT-004.
-

114. Restricciones de Fase

Mientras RPC-002 permanezca en Draft o In Review:

- No se iniciará RPC-003.
- No se iniciará Business Discovery.

- No se iniciará Domain Discovery.
- No se identificarán Bounded Contexts.
- No se diseñarán microservicios.
- No se tomarán decisiones técnicas de implementación.
- No se generará código productivo.

115. Próximo Gate

El gate actual será:

RPC-002
AI Collaboration Charter

Draft
↓
Review
↓
Approved

Su aprobación habilitará:

RPC-003 — Architecture Governance

116. Relación con otros documentos

Documento superior

- RPC-000 — Blueprint Manifest

Constitución gobernante

- RPC-001 — AI Project Constitution

Antecedente fundacional

- BOOT-004 — AI Collaboration Model

Templates aplicables

- TPL-000 — Document Template Standard

- TPL-003 — Prompt Template

Documento siguiente condicionado

- RPC-003 — Architecture Governance

Documentos futuros relacionados

- RPC-004 — Business Discovery Guide
- RPC-005 — Domain Modeling Guide
- RPC-016 — Prompt Engineering Guide
- PRM Series
- CHK Series

117. Principio Final

La colaboración humano-IA del proyecto seguirá:

La IA debe hacer al equipo más capaz, no menos responsable.

Una colaboración exitosa no se medirá por cuánto trabajo se delegó.

Se medirá por cuánto mejoró:

- La calidad del razonamiento.
- La velocidad de aprendizaje.
- La trazabilidad.
- La consistencia.
- La capacidad de evolucionar el sistema.

La inteligencia artificial deberá convertirse en una extensión disciplinada del proceso de ingeniería, nunca en un proceso paralelo fuera del gobierno del proyecto.

118. Historial de Cambios

Versión	Fecha	Descripción
1.0	2026-07-24	Creación inicial de RPC-002 — AI Collaboration Charter. Documento generado en estado Draft para revisión.

Estado

Draft — Pending Review

No se autoriza el inicio de **RPC-003 — Architecture Governance** hasta completar la revisión y aprobación de RPC-002.